

# Programação de Sistemas

Acesso a arquivos e alocação dinâmica de structs em C

Marcelo de Gomensoro Malheiros



# Recapitulando...

» Funções *sizeof*, *malloc* e *free*



- *sizeof*: retorna o número de bytes ocupado por um tipo
- *malloc*: reserva memória com o tamanho dado, em bytes
- *free*: libera a área de memória vinculada a um ponteiro
  - `#include <stdlib.h>`
  - `int main(void) {`
  - `int *p_i = malloc(10 * sizeof(int));`
  - `p_i[0] = 123;`
  - `// ...`
  - `free(p_i);` // *p\_i não pode ser usado depois!*
  - `p_i = NULL;` // *recomenda-se zerar o ponteiro*

# Recapitulando...



» Retorno de vetor alocado dinamicamente

- Vetores só podem ser retornados se alocados dinamicamente
  - `int *cria_vetor(int tamanho) {`
  - `int *p_i = malloc(tamanho * sizeof(int)); // alocação dinâmica`
  - `// inicializa o vetor`
  - `for (int c = 0; c < tamanho; c++) {`
  - `p_i[c] = 1;`
  - `}`
  - `return p_i; // retorna o vetor`
  - `}`

# Acesso a arquivos

- Um arquivo em modo **texto** pode ser escrito assim:
  - Criação em modo de escrita (*write*). O conteúdo anterior é apagado:
  - `FILE *arq = fopen("arquivo.txt", "w");`
  - Escreve *string* ou formato para dentro do arquivo `arq`:
  - `fprintf(arq, "Isso é um teste!\n");`
  - `fprintf(arq, "palavra\n");`
  - `fprintf(arq, "%i %f\n", 1, 2.34);`
  - Fecha arquivo e zera ponteiro (por segurança):
  - `fclose(arq);`
  - `arq = NULL;`

- Um arquivo em modo **texto** pode ser lido assim:
  - `FILE *arq = fopen("arquivo.txt", "r");` // abertura
  - `char linha[100 + 1];`
  - `while (1) {` // repete sempre
  - `fscanf(arq, "%100[^\n]*c", linha);` // leitura linha a linha
  - `if (feof(arq)) { break; }` // final de arquivo
  - `printf("%s\n", linha);`
  - `}`
  - `fclose(arq);` // fecha arquivo
  - `arq = NULL;` // zera ponteiro

- É recomendável checar se a criação ou a abertura do arquivo tiveram sucesso:
  - `FILE *arq = fopen("arquivo.txt", "r");`
  - `if (arq == NULL) {`
  - `printf("A abertura do arquivo falhou!\n");`
  - `}`
  - `else {`
  - `// ...`
  - `}`

- A biblioteca de funções de C provê apenas as operações mais gerais de leitura e escrita de arquivos:
  - `fopen`, `fprintf`, `fscanf`, `feof`, `fclose`
- Outras operações de manipulação de arquivos podem ser feitas em C, mas são específicas ao sistema operacional:
  - copiar, renomear, obter data, remover, criar diretório, etc...

# Alocação dinâmica de *structs*

# Alocação dinâmica de *structs*

» Criação



- *Structs* também podem ser alocadas dinamicamente
- Definição:
  - `typedef struct {`
  - `char nome[20];`
  - `int idade;`
  - `} Pessoa;`
- A alocação dinâmica cria uma *struct* não-inicializada
- Exemplo:
  - `Pessoa *p_p =`
  - `malloc(sizeof(Pessoa));`
  - `// ...`
  - `free(p_p);`
  - `p_p = NULL;`

# Alocação dinâmica de *structs*

## » Acesso



- O acesso aos membros pode ser feito combinando os operadores `*` e `.`
  - `(*p_p).idade = 20;`
  - `printf("%i", (*p_p).idade);`
- C tem uma notação alternativa para isso, usando o operador `->`
  - `p_p->idade = 20;`
  - `printf("%i", p_p->idade);`

# Alocação dinâmica de *structs*

## » Vetores de *structs*



- Naturalmente, a alocação dinâmica também possibilita a criação de vetores de *structs* não-inicializadas
- Exemplo:
  - `Pessoa *pessoas = malloc(10 * sizeof(Pessoa));`
  - `// ...`
  - `strcpy(pessoas[0].nome, "Pedro");`
  - `pessoas[0].idade = 13;`
  - `// ...`
  - `free(pessoas);`
  - `pessoas = NULL;`

# Exercícios

- Faça um programa que cria um arquivo `planilha.csv`. Comece com o valor `x = 0` e incremente em 1 até chegar a 10. Em cada valor, imprima no arquivo o número, sua raiz quadrada e também  $\sin(x)$ , separados por uma vírgula.
- Defina uma *struct* chamada `Retangulo` e crie uma função que aloca dinamicamente, inicializa e devolve um retângulo de lados 1 e 4. Crie também uma função para calcular a área e outra para transformá-lo em um quadrado de mesma área.

**Perguntas?**